

GPT-5.5 Direct-Edit Quality Frontier

Reasoning-level interactions between GPT-5.5 root integrators and GPT-5.5 implementation leaves

By Adam Owada, with Codex
June 30, 2026

Abstract

This paper reports an 80-run RuleLedger v3 experiment that replaced Spark implementation leaves with GPT-5.5 direct-edit leaves. The study focused on the quality frontier rather than a complete low-through-xhigh grid: 20 runs each for high root plus high leaves, xhigh root plus high leaves, high root plus xhigh leaves, and xhigh root plus xhigh leaves.

The main result was not the expected "xhigh everywhere" outcome. The strongest observed mean quality came from the simplest high-tier team: high root plus high GPT-5.5 leaves scored 0.769 mean quality with 5.059M mean implementation tokens. That narrowly exceeded the prior solo GPT-5.5 xhigh baseline (0.755) and prior xhigh Spark direct baseline (0.734), but at materially higher token cost. Xhigh root plus xhigh leaves produced the best single run (0.989) and the best top-quartile mean (0.974), but its overall mean was lower (0.732) and its mean implementation cost was much higher (6.777M tokens).

The practical conclusion is sharp: if GPT-5.5 direct-edit leaves are used, high root plus high leaves is the best observed configuration in this study. Xhigh reasoning still matters for peak runs, but raising the root or leaves to xhigh did not improve mean quality enough to justify the coordination cost on RuleLedger v3.

Decision Summary

Developer goal	Recommended configuration	Why
Highest observed mean quality in this experiment	High root + high GPT-5.5 leaves	Best mean quality (0.769), best median (0.850), lowest token cost among the four GPT-5.5 leaf cells
Highest single-run ceiling	Xhigh root + xhigh GPT-5.5 leaves	Best individual run (0.989) and best top-quartile mean (0.974)
Best practical GPT-5.5 leaf setting	High root + high GPT-5.5 leaves	It beat xhigh/high by +0.065 , high/xhigh by +0.085 , and xhigh/xhigh by +0.037 observed mean quality
Baseline if cost matters more than marginal frontier quality	Solo GPT-5.5 high or solo GPT-5.5 xhigh	Prior solo baselines remain much more token-efficient; the GPT-5.5 leaf topology spends more to chase a small mean-quality gain
Diagnostic next run	Fewer high GPT-5.5 leaves, or a high root with stricter integration constraints	The full six-leaf topology appears to create coordination load; xhigh leaves did not help the high root

Terminology

Term	Meaning
Root integrator	The GPT-5.5 agent that plans, delegates, reviews leaf diffs, and submits the final measured implementation.
GPT-5.5 leaf	A GPT-5.5 worker assigned a focused implementation, test, parity, or adversarial review slice.
Direct-edit leaf	A leaf running in an isolated writable workspace. The root later inspects the leaf diff and chooses what to integrate.
Reasoning level	A runtime reasoning-effort setting in the benchmark harness: high or xhigh in this experiment.

Term	Meaning
Root tokens	GPT-5.5 implementation tokens attributed to the root planning and integration stages.
Leaf tokens	GPT-5.5 implementation tokens attributed to leaf stages.
Total implementation tokens	Root tokens plus leaf tokens. Judge tokens are excluded unless explicitly noted.
Quality	The 0 to 1 RuleLedger v3 composite score weighted toward hidden correctness, parity, performance, and judge review.

All implementation tokens in this experiment are GPT-5.5 tokens. The harness still uses a field named `spark_mode` for the direct-edit topology, but the leaf model in this study is GPT-5.5, not Spark.

Benchmark Task

RuleLedger v3 is a subscription-ledger implementation benchmark with both TypeScript and Python surfaces. A measured run starts from a frozen starter project and must implement one coherent replay model across event parsing, normalization, account lineage, billing, deterministic reports, parity outputs, compatibility APIs, and performance-sensitive workloads.

The task rewards global consistency. A local patch can pass public tests while hidden checks still catch parity drift, report mismatches, incomplete merge lineage, incorrect correction semantics, or repeated full-history scans. This is why RuleLedger v3 is a useful stress test for subagent topology: extra workers can expand local search, but the final answer still has to integrate into one semantics model.

Experiment Design

The official study contains 80 scored implementation runs:

Cell	Root model	Root reasoning	Leaf model	Leaf reasoning	Leaves	Mode	Runs
GQF0	GPT-5.5	high	GPT-5.5	high	6	direct edit	20
GQF1	GPT-5.5	xhigh	GPT-5.5	high	6	direct edit	20
GQF2	GPT-5.5	high	GPT-5.5	xhigh	6	direct edit	20
GQF3	GPT-5.5	xhigh	GPT-5.5	xhigh	6	direct edit	20

The topology was staged:

Frozen RuleLedger v3 starter

-> GPT-5.5 root planning stage

-> six GPT-5.5 direct-edit leaf stages

-> GPT-5.5 root integration stage

-> public tests, hidden tests, scoring, judge review, usage parsing

The leaf roles matched the previous staged direct-edit design:

Leaf	Responsibility
1	TypeScript parsing, normalization, views, and migration compatibility
2	TypeScript replay, billing, reporting, performance, and API integration
3	Python parsing, normalization, views, and migration compatibility
4	Python replay, billing, reporting, performance, and API integration

Leaf	Responsibility
5	Cross-language parity, fixtures, public tests, and regression review
6	Adversarial review for localization, maintainability, performance, and hidden-test risk

Measured implementation runs used `codex exec --json`. The full run used `-Jobs 7` and `-JudgeJobs 6`. The selected matrix, benchmark template, hidden cases, scoring config, prompt templates, judge schema, model identifiers, and timeouts were frozen in the run metadata. The judge was GPT-5.5 xhigh for every cell.

Execution Audit

The final experiment directory is:

```
runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6
```

Validation passed after the rerun workflow:

Check	Result
Matrix contract	80 configured runs, 20 per cell
Preflight	passed
Experiment metadata	passed
Run artifacts	validated for 80 runs
Hidden-test isolation	validated for 80 runs
Judge JSON	validated for 80 runs
Resume contract	passed
Report outputs	CSV, SQLite, aggregate JSON, HTML, and PDF validated

The first full pass produced five transient root integration stream failures with `os error 10065`. Those runs were rerun with the resume/rerun-failed path. The final scored dataset has 80/80 rows, zero implementation failures, zero judge failures, and `per_event_model` token attribution for every row. No usage limit interruption was observed.

Measurement

Reported values use fixed rounding: quality-like values are rounded to three decimals, token counts use whole-token means or `M` shorthand, and efficiency ratios use three significant figures in scientific notation.

Quality

Quality is the RuleLedger v3 `reasoning_ladder_v3` composite:

Component	Weight	Meaning
Hidden correctness	0.50	Correctness on frozen private RuleLedger v3 behavior checks, excluding parity and performance categories
Hidden parity	0.15	TypeScript/Python agreement on hidden parity checks
Performance	0.10	Performance behavior on hidden workloads
Judge	0.22	GPT-5.5 xhigh review of source, diffs, logs, public checks, and hidden-result summaries
Minimality	0.03	Production LOC minimality signal relative to the configured target

The judge saw implementation evidence and hidden-result summaries, but not the private hidden case payloads.

Tokens

The harness parsed `turn.completed.usage` from JSONL event streams and used per-event model attribution. Because both root and leaves were GPT-5.5, total implementation tokens and GPT-5.5 implementation tokens are the same total. Root and leaf token fields are still useful because they expose coordination cost.

Efficiency tables use ratio of means:

```
mean(quality) / mean(tokens)
```

This matches aggregate budget planning better than averaging per-run ratios.

Results At A Glance

Cell	Runs	Quality mean	Median	Hidden correctness	Judge	Root tokens	Leaf tokens	Total tokens	Primary read
high root + high leaves	20	0.769	0.850	0.739	0.757	3.397M	1.662M	5.059M	Best observed mean, best practical cell
xhigh root + high leaves	20	0.705	0.650	0.678	0.729	4.274M	1.635M	5.909M	More root cost, lower mean
high root + xhigh leaves	20	0.684	0.611	0.659	0.646	3.551M	2.263M	5.814M	Diagnostic cell underperformed
xhigh root + xhigh leaves	20	0.732	0.682	0.709	0.747	4.565M	2.212M	6.777M	Best single run, not best mean

High/high is the surprise winner. It is the cheapest of the four GPT-5.5 leaf cells and has the highest mean, median, hidden correctness, hidden parity, performance, judge score, and implementation efficiency.

Frontier And Tail Behavior

Cell	Top quartile mean	Bottom quartile mean	Best	Worst	>=0.8	>=0.9	> prior solo xhigh mean (0.755)
high root + high leaves	0.960	0.538	0.972	0.441	11/20	7/20	12/20
xhigh root + high leaves	0.943	0.573	0.976	0.565	5/20	3/20	5/20
high root + xhigh leaves	0.904	0.529	0.967	0.513	6/20	1/20	7/20
xhigh root + xhigh leaves	0.974	0.564	0.989	0.530	6/20	5/20	7/20

Xhigh/xhigh remains a ceiling candidate: it produced the best individual run and the strongest top quartile. But high/high had more consistently strong runs and the best mean. The high/high median of `0.850` is particularly important: more than half of the high/high runs cleared a level that prior studies treated as strong RuleLedger v3 performance.

Comparison To Prior Baselines

The relevant prior baselines are reported in the RuleLedger v3 reasoning paper and the Spark mode efficiency paper:

Prior configuration	Runs	Mean quality	Mean implementation tokens
Solo GPT-5.5 high	50	0.696	2.612M
Solo GPT-5.5 xhigh	50	0.755	3.334M
High root + xhigh Spark direct leaves	20	0.712	3.433M
Xhigh root + xhigh Spark direct leaves	20	0.734	4.488M

Against those baselines:

New cell	Quality	Tokens	vs solo high	vs solo xhigh	vs Spark high direct	vs Spark xhigh direct
high root + high GPT-5.5 leaves	0.769	5.059M	+0.073	+0.014	+0.057	+0.035
xhigh root + high GPT-5.5 leaves	0.705	5.909M	+0.009	-0.050	-0.007	-0.029
high root + xhigh GPT-5.5 leaves	0.684	5.814M	-0.012	-0.071	-0.028	-0.050
xhigh root + xhigh GPT-5.5 leaves	0.732	6.777M	+0.036	-0.023	+0.020	-0.002

High/high is the only new cell that improves on the prior solo xhigh mean. The gain is small (+0.014) and should be read as an observed frontier improvement, not a settled separation claim. It costs +1.725M more implementation tokens than solo xhigh and +0.570M more than the prior xhigh Spark direct cell.

Efficiency

Cell	Mean quality	Mean implementation tokens	Quality / implementation token	Quality / judge-inclusive token
high root + high leaves	0.769	5.059M	1.52e-7	1.38e-7
xhigh root + high leaves	0.705	5.909M	1.19e-7	1.11e-7
high root + xhigh leaves	0.684	5.814M	1.18e-7	1.10e-7
xhigh root + xhigh leaves	0.732	6.777M	1.08e-7	1.00e-7

All GPT-5.5 leaf cells are less token-efficient than the prior solo high (2.66e-7) and solo xhigh (2.26e-7) baselines. The experiment is therefore a quality-frontier result, not a token-efficiency result.

Within the new 2x2 matrix, high/high dominates: it has the highest quality and the lowest token cost. Xhigh/xhigh has the highest peak but the lowest efficiency.

Pairwise Effects

Bootstrap intervals below are approximate 95% intervals over the 20-run cells. They are descriptive, not preregistered inferential tests.

Comparison	Mean quality delta	Approx. 95% bootstrap interval	Interpretation
Xhigh root effect with high leaves	-0.065	[-0.162, +0.038]	Xhigh root did not help high leaves
Xhigh root effect with xhigh leaves	+0.048	[-0.048, +0.146]	Xhigh root helped xhigh leaves directionally

Comparison	Mean quality delta	Approx. 95% bootstrap interval	Interpretation
Xhigh leaf effect with high root	-0.085	[-0.187, +0.019]	Xhigh leaves hurt the high root in the observed mean
Xhigh leaf effect with xhigh root	+0.027	[-0.068, +0.121]	Xhigh leaves helped xhigh root directionally
Xhigh/xhigh vs high/high	-0.037	[-0.141, +0.070]	Xhigh/xhigh was not the mean-quality winner

The factorial read is also sobering:

Factor	Quality main effect	Total-token main effect	Root-token main effect	Leaf-token main effect
Root xhigh minus root high	-0.008	+0.907M	+0.946M	-0.039M
Leaf xhigh minus leaf high	-0.029	+0.811M	+0.223M	+0.588M

On average across the 2x2, raising reasoning to xhigh increased token spend without increasing mean quality. The interaction matters: xhigh root appears more able to use xhigh leaf work than high root does, but the combined cell still did not beat high/high on mean.

Hypothesis Audit

Hypothesis 1: Xhigh Root + Xhigh Leaves Is The Raw Quality Favorite

Result: partially supported for peak quality, not supported for mean quality.

Xhigh/xhigh produced the best single run (0.989) and best top-quartile mean (0.974). That supports the idea that the all-xhigh topology can reach the highest ceiling.

But the average result contradicts the hypothesis. Xhigh/xhigh mean quality was 0.732, below high/high at 0.769. It also cost 6.777M implementation tokens, the highest of the four cells. The xhigh/xhigh cell changed slightly more files (12.45 vs 11.40 for high/high), produced much more test LOC (84 vs 29), and had weaker minimality (0.719 vs 0.825). That pattern is consistent with coordination cost and possible over-editing.

Hypothesis 2: Xhigh Root + High Leaves Might Be The Best Practical Cell

Result: not supported.

Xhigh/high scored 0.705, below high/high by 0.065 mean quality while using 0.850M more implementation tokens. It did have a healthier weak-run floor than high/high: no xhigh/high run fell below 0.5, while high/high had one 0.441 run. But the central tendency and efficiency both favor high/high.

The likely read is that the xhigh root spent more context and tokens integrating leaf work without extracting enough additional value from high leaves. In this topology, xhigh root reasoning was not automatically a better integrator.

Hypothesis 3: High Root + Xhigh Leaves Is Diagnostic

Result: strongly diagnostic, and it points toward integration bottleneck.

High/xhigh scored the lowest observed mean (0.684) despite spending 2.263M leaf tokens per run. It also had the lowest judge mean (0.646). If leaf implementation quality were the dominant limiting factor, xhigh leaves should have lifted the high root. Instead, high/high beat high/xhigh by +0.085.

The companion interaction is useful: xhigh/xhigh beat high/xhigh by +0.048. That suggests xhigh leaf work may require an xhigh root to reconcile it. The diagnosis is not "xhigh leaves are bad"; it is "stronger leaves increase the integration burden, and a high root did not reliably convert that extra work into coherent final implementations."

Hypothesis 4: High Root + High Leaves Is The Economical Upper-Mid Baseline

Result: understated. It was not merely the economical baseline; it was the observed winner.

High/high had the best mean quality (0.769), best median (0.850), strongest hidden correctness (0.739), best parity and performance means (0.833 each), and lowest total implementation tokens (5.059M) among the four GPT-5.5 leaf cells. It is the practical default if this direct-edit GPT-5.5 leaf topology is used again.

The caveat is external efficiency. High/high is economical only relative to the other GPT-5.5 leaf cells. It remains more expensive than solo GPT-5.5 high and solo GPT-5.5 xhigh.

Detailed Findings

High Root + High Leaves

This cell was the cleanest result. It scored 0.769 mean quality and 0.850 median quality, with 12/20 runs above the prior solo xhigh mean of 0.755. It also had the best hidden correctness, hidden parity, performance, and judge means in the new matrix.

The result is not variance-free. One run scored 0.441, and the bottom quartile mean was 0.538. But the upper half was strong enough to carry the best mean: the top quartile averaged 0.960, and seven runs reached at least 0.9 quality.

The main practical point is that this was both the highest-quality and lowest-cost GPT-5.5 leaf cell. It used 3.397M root tokens and 1.662M leaf tokens per run, for 5.059M total implementation tokens.

Xhigh Root + High Leaves

Xhigh/high underperformed the practical hypothesis. It scored 0.705 mean quality, with a 0.650 median. It used 4.274M root tokens, roughly 877k more root tokens than high/high, while leaf tokens stayed similar.

The cell had no catastrophic low tail: every run scored at least 0.565. But it produced fewer strong runs than high/high. Only 5/20 runs exceeded the prior solo xhigh mean, compared with 12/20 for high/high.

This is evidence that xhigh integration is not automatically better when leaf outputs are already high-quality. Extra root reasoning may create more review, rewrite, or reconciliation work without improving the submitted semantics.

High Root + Xhigh Leaves

High/xhigh was the weakest cell. It scored 0.684 mean quality and 0.611 median quality. It spent 2.263M leaf tokens per run, 600k more leaf tokens than high/high, but did not convert that extra leaf effort into final quality.

This cell is the strongest diagnostic result. If better leaves were the main bottleneck, high/xhigh should have been competitive. Instead, it fell below solo GPT-5.5 high and below both prior Spark direct baselines. The high root appears to have struggled to integrate or select from more expansive xhigh leaf diffs.

Xhigh Root + Xhigh Leaves

Xhigh/xhigh is the ceiling cell but not the default. It produced the best run (0.989) and the best top-quartile mean (0.974). It also had the second-best mean hidden correctness (0.709) and second-best judge mean (0.747).

The cost side is harsh. It used 4.565M root tokens and 2.212M leaf tokens per run, for 6.777M total implementation tokens. That is +1.718M tokens relative to high/high and +3.443M relative to prior solo xhigh. It also had weaker median quality (0.682) than high/high.

This cell is useful when searching for maximum possible runs, but not as the mean-quality or practical-cost recommendation.

Why High/High May Have Won

The RuleLedger v3 task needs coherent shared semantics more than independent local patch volume. Six direct-edit leaves already introduce a lot of candidate code. High reasoning may be enough for each leaf to make useful progress while keeping changes compact enough for the root to integrate.

Raising leaves to xhigh likely increased the breadth and ambition of leaf patches. That can improve peak outcomes, but it also increases root selection and merge burden. Raising the root to xhigh increased root tokens and production LOC, but did not reliably improve final coherence. The best mean result came from the cell with enough reasoning to do the work and the least extra coordination pressure.

This interpretation also matches prior Spark findings. Spark direct edit helped more at medium and high than at xhigh; at xhigh, the solo model's single-thread coherence was hard to beat. GPT-5.5 leaves are stronger than Spark leaves, but stronger leaves do not remove the cost of synthesis.

Implications

1. GPT-5.5 direct-edit leaves can move the observed quality frontier. High/high narrowly beat the prior solo xhigh mean.

1. The win is not token-efficient. The same cell used 5.059M implementation tokens per run, versus 3.334M for solo xhigh.

1. Xhigh should not be applied uniformly. In this matrix, xhigh reasoning added cost faster than it added mean quality.

1. Integration is the bottleneck to study next. The high/xhigh diagnostic cell implies that more capable leaves are not enough unless the root can reconcile their work into one implementation model.

1. The next practical topology should probably reduce coordination load: fewer leaves, stricter leaf output contracts, narrower diffs, or root integration prompts that force explicit selection and rejection of leaf changes.

Limitations

- The benchmark is RuleLedger v3. Results may differ for tasks that are more modular, easier to partition, or less parity-sensitive.
- Each GPT-5.5 leaf cell has 20 runs. The prior solo comparison has 50 runs per reasoning level, and the prior Spark direct comparisons have 20 runs per relevant cell.
- The experiment changes two things relative to prior Spark direct runs: leaf

model and leaf reasoning policy. It answers the practical GPT-5.5 leaf quality-frontier question, not a clean model-only causal question.

- All GPT-5.5 leaf cells used six leaves. Smaller teams may keep the quality gain while reducing coordination cost.

- All leaves shared one reasoning level per cell. Mixed policies, such as high implementers with xhigh adversarial review, remain untested.

- The judge is GPT-5.5 xhigh. It is held fixed and topology-blind, but it is still an LLM review signal rather than independent human adjudication.

- Elapsed seconds are batch harness measurements under local scheduling and high parallelism. They should not be read as interactive latency estimates.

- The first full pass had five transient stream failures that were rerun. The

final validation passed for all 80 selected runs, but operational robustness remains part of the cost profile for large experiments.

Appendix A: Detailed Quality Table

Cell	Runs	Quality mean	Median	SD	Min	Q1	Q3	Max	Hidden correctness	Hidden parity	Performance	Judge	Minimality
high root + high leaves	20	0.769	0.850	0.182	0.441	0.591	0.924	0.972	0.739	0.833	0.833	0.757	0.825
xhigh root + high leaves	20	0.705	0.650	0.151	0.565	0.587	0.763	0.976	0.678	0.733	0.733	0.729	0.724
high root + xhigh leaves	20	0.684	0.611	0.158	0.513	0.563	0.874	0.967	0.659	0.750	0.750	0.646	0.841
xhigh root + xhigh leaves	20	0.732	0.682	0.165	0.530	0.598	0.897	0.989	0.709	0.767	0.767	0.747	0.719

Appendix B: Detailed Token Table

Cell	Root tokens	Leaf tokens	Total implementation tokens	Judge tokens	Judge-inclusive tokens	Elapsed sec	Changed files	Production LOC	Test LOC
high root + high leaves	3,396,713	1,662,234	5,058,946	522,427	5,581,373	1,153	11.4	1,863	29
xhigh root + high leaves	4,273,810	1,635,473	5,909,283	434,141	6,343,424	1,489	12.2	2,304	12
high root + xhigh leaves	3,551,053	2,262,508	5,813,562	435,974	6,249,536	1,398	11.4	1,742	19
xhigh root + xhigh leaves	4,564,985	2,212,050	6,777,035	520,771	7,297,806	1,731	12.4	2,324	84

Appendix C: Reproducibility

Key source files:

- Experiment plan: `plans/experiments/gpt55-direct-quality-frontier.md`
- Full config: `configs/gpt55_direct_quality_frontier.yaml`
- Pilot config: `configs/gpt55_direct_quality_frontier_pilot.yaml`
- Shared staged prompt: `prompts/task_common_staged_leaf_v3.md`
- GPT-5.5 staged leaf prompt: `prompts/task_staged_gpt55_leaf_v3.md`
- Judge prompt: `prompts/judge.md`
- Scoring profile: `configs/scoring_v3.yaml`

Run command:

```
.\scripts\run_experiment.ps1 `
-Config configs/gpt55_direct_quality_frontier.yaml `
-Jobs 7 `
-JudgeJobs 6 `
-ExperimentName gpt55_frontier_j7_j6
```

Rerun command for transient failures:

```
.\scripts\run_experiment.ps1 `
-Config configs/gpt55_direct_quality_frontier.yaml `
-Jobs 7 `
-JudgeJobs 6 `
-Resume C:\Users\adams\projects\codex-subagent-testing\runs\20260629T224027-gpt55_direct_quality_frontier-
gpt55_frontier_j7_j6 `
-RerunFailed
```

Frozen run metadata:

Field	Value
Repository branch	<code>feature/gpt-subagent-testing</code>
Repository head	<code>427623f9e5372d7c9f7ef11b9aa94f8f776778c2</code>
Config SHA-256	<code>51053ea9713a01929e0ddab4d66b829034dc27a76048535e3445f39fc5ddd6b6</code>
Full matrix SHA-256	<code>21ffa2543e0b2e8c1de22ed4679edd9495eeff1580392c3d35870e375f113bbd</code>
Benchmark template SHA-256	<code>edfde6362042bfdf100bedfe63c273c817072004cc42ff3952ad3afb9c319699</code>
Hidden cases SHA-256	<code>b7f42505d0503883a3e8dc001d41a16254b67d1aa3aea1fd337acb5e620a8d6b</code>
Hidden manifest SHA-256	<code>63937005528b0ff1842bcd10a57617da49acbbbc0e4b67dfa8b5f65547ecb362</code>
Scoring config SHA-256	<code>dc50227fd818efc567b756bc2be2bf6bf2a22850072bd7ab14ca6673c68cb76c</code>

Generated evidence:

- Results CSV: `runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6/results/results.csv`
- Aggregate JSON: `runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6/results/aggregate.json`

- SQLite results: `runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6/results/results.sqlite`
 - HTML report: `runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6/report/report.html`
 - PDF report: `runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6/report/report.pdf`
 - Validation: `runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6/validation.json`
-

Appendix D: References

- RuleLedger v3 reasoning white paper: `papers/ruleledger-v3-white-paper.md`
- Spark mode efficiency white paper: `papers/spark-mode-efficiency-white-paper.md`
- RuleLedger v3 issue brief: `benchmark_template_v3/docs/ruleledger_v3_issue_brief.md`
- RuleLedger v3 architecture notes: `benchmark_template_v3/docs/ruleledger_v3_architecture.md`